

TECHNOVA SERVICES

Projet Annuel 4SRC 2025-2026 · Phase 3

VM-DOCKER-01

Environnement conteneurisé · Docker Engine · Docker Compose · Portainer

Antoine Accault

Chef de projet · Partie Systèmes

ESGI Mastère SRC · Année 2025-2026

Table des matières

1. Présentation générale	3
1.1 Fiche technique	3
2. Installation et configuration	3
2.1 Docker Engine officiel	3
3. Gestion des utilisateurs	4
3.1 Séparation des comptes	4
3.2 Accès SSH	4
4. Stack applicatif webapp	4
4.1 Architecture	5
4.2 docker-compose.yml	5
4.3 Gestion des secrets	6
5. Fonctionnalités démontrées	6
5.1 DNS interne Docker	6
6. Portainer	6
6.1 Justification du choix	6
6.2 Accès sécurisé	7
7. Accès aux services applicatifs	7
8. Justification des choix techniques	8
8.1 Docker Engine vs Podman	8
8.2 Docker Compose vs docker run	8
8.3 nginx Alpine + MariaDB + phpMyAdmin	8
9. Sécurité	9

1. Présentation générale

La VM Docker-01 héberge l'environnement de conteneurisation de TechNova. Elle exécute Docker Engine (runtime) et Docker Compose (orchestration), avec Portainer comme interface web de supervision.

1.1 Fiche technique

Hyperviseur :

Proxmox VE 9 (cluster 3 nœuds)

VMID :

108

Système d'exploitation :

Ubuntu Server 24.04 LTS

Nom d'hôte :

docker

Adresse IP :

192.168.20.30/24

Passerelle :

192.168.20.1

DNS :

192.168.20.20 (AD-01)

Ressources :

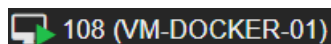
4 vCPU · 4 Go RAM · 40 Go de disque

Docker Engine :

28.x (repo officiel Docker)

Docker Compose :

v2.x (plugin)



2. Installation et configuration

2.1 Docker Engine officiel

Docker Engine a été installé depuis le repository officiel Docker (et non depuis podman-docker) pour garantir la compatibilité totale avec l'écosystème Docker :

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
```

```
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" |
```

```
sudo tee /etc/apt/sources.list.d/docker.list
```

```
sudo apt update
```

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
```

```
technova@docker:/opt/docker-stacks/webapp$ docker compose version
Docker Compose version v5.3.1
technova@docker:/opt/docker-stacks/webapp$ docker --version
Docker version 29.6.2, build dfc4efb
technova@docker:/opt/docker-stacks/webapp$
```

3. Gestion des utilisateurs

3.1 Séparation des comptes

Deux comptes utilisateurs sont configurés :

technova :

Compte administrateur dédié, distinct du user quotidien

Les deux comptes sont membres des groupes sudo et docker. Le compte root n'est jamais utilisé en connexion directe SSH (PermitRootLogin no).

groups technova

```
# → technova adm cdrom sudo dip plugdev lxd docker
```

3.2 Accès SSH

La connexion se fait via ProxyJump automatique par le Bastion :

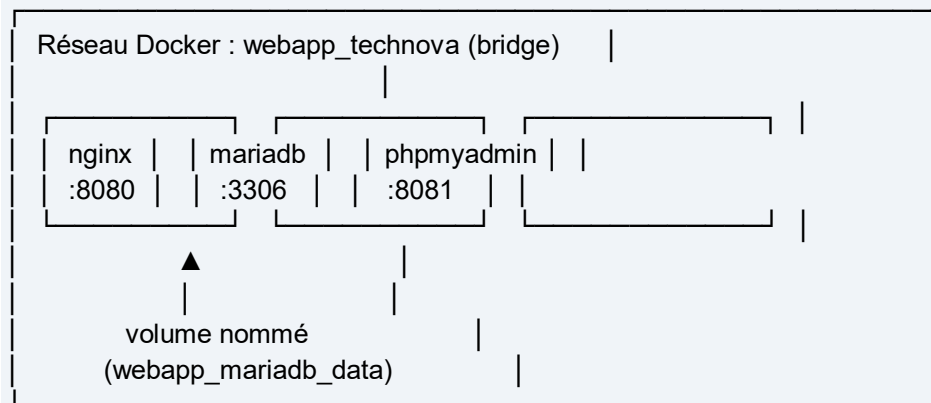
```
# Configuration ~/.ssh/config
Host docker
  HostName 192.168.20.30
  User technova
  ProxyJump bastion
  IdentityFile ~/.ssh/id_ed25519

# Commande
ssh docker-admin
```

4. Stack applicatif webapp

Un stack applicatif de démonstration est déployé via Docker Compose dans /opt/docker-stacks/webapp/. Il comprend trois services interconnectés :

4.1 Architecture



4.2 docker-compose.yml

```
services:
  nginx:
    image: nginx:alpine
    container_name: technova-web
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
    restart: unless-stopped
    networks:
      - technova

  mariadb:
    image: mariadb:11
    container_name: technova-db
    environment:
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
      MARIADB_DATABASE: ${MARIADB_DATABASE}
      MARIADB_USER: ${MARIADB_USER}
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}
    volumes:
      - mariadb_data:/var/lib/mysql
    restart: unless-stopped
    networks:
      - technova

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: technova-pma
    ports:
      - "8081:80"
    environment:
      PMA_HOST: mariadb
      PMA_USER: ${PMA_USER}
      PMA_PASSWORD: ${PMA_PASSWORD}
    depends_on:
      - mariadb
    restart: unless-stopped
```

```
networks:
  - technova

volumes:
  mariadb_data:
networks:
  technova:
    driver: bridge
```

4.3 Gestion des secrets

Les mots de passe sont externalisés dans un fichier .env avec permissions restrictives (chmod 600) :

```
# /opt/docker-stacks/webapp/.env
MARIADB_ROOT_PASSWORD=TechNova2026!
MARIADB_DATABASE=technova_db
MARIADB_USER=technova
MARIADB_PASSWORD=User2026!
PMA_USER=root
PMA_PASSWORD=TechNova2026!
```

Sécurité :

- Permissions 600 (lisible uniquement par le propriétaire)
- Exclu de Git via .gitignore
- Le YAML est safe à commiter (aucun secret en dur)

5. Fonctionnalités démontrées

5.1 DNS interne Docker

Les conteneurs se résolvent par nom de service grâce au DNS embarqué de Docker :

```
docker compose exec nginx ping -c 2 mariadb
# → 64 bytes from mariadb.webapp technova (172.18.0.X)
technova@docker:/opt/docker-stacks/webapp$ docker compose exec nginx ping -c 2 mariadb
PING mariadb (172.18.0.4): 56 data bytes
64 bytes from 172.18.0.4: seq=0 ttl=64 time=0.090 ms
64 bytes from 172.18.0.4: seq=1 ttl=64 time=0.046 ms

--- mariadb ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.046/0.068/0.090 ms
technova@docker:/opt/docker-stacks/webapp$
```

6. Portainer

Portainer CE est déployé comme interface web de gestion Docker, complémentaire à la ligne de commande.

6.1 Justification du choix

- Open source (BSD-2 clause)
- Interface intuitive pour les opérateurs non-experts en CLI

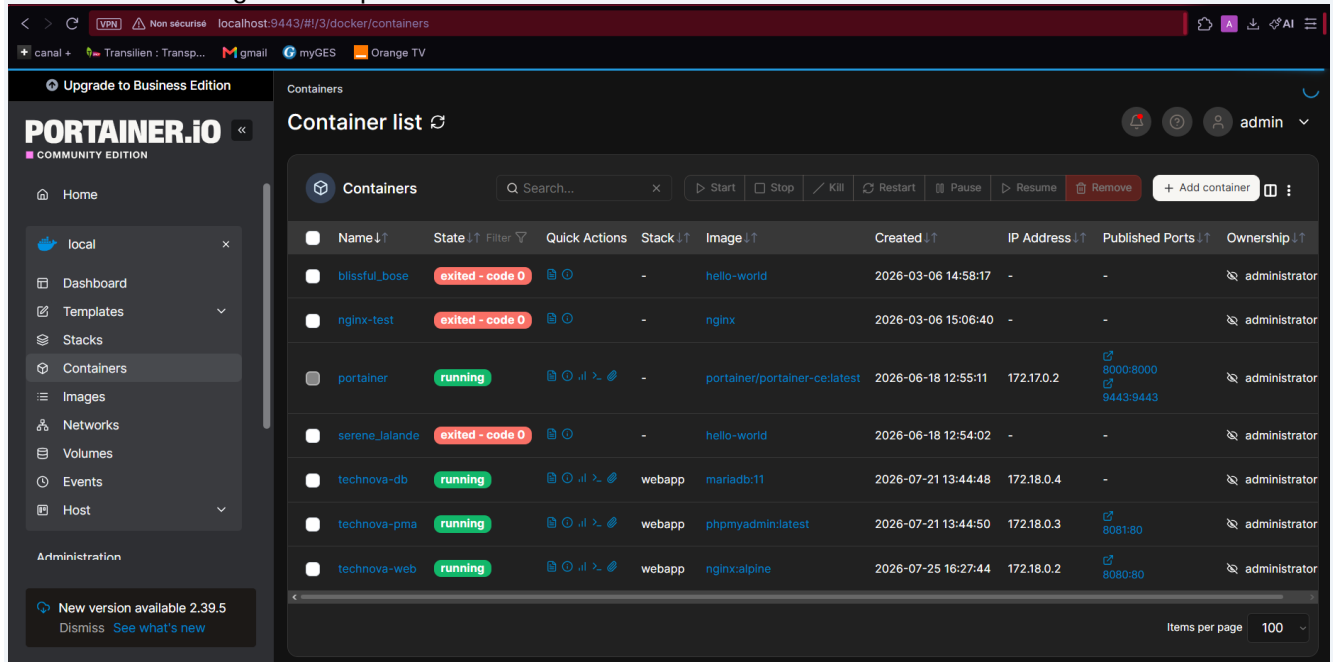
- Support natif Docker Compose (import de stacks)
- Multi-environnements (Docker, Swarm, Kubernetes) — évolutif
- Léger (~250 Mo image, ~100 Mo RAM)

6.2 Accès sécurisé

Portainer est exclusivement accessible via tunnel SSH depuis le Bastion :

```
ssh -p 2222 -L 9443:192.168.20.30:9443 technova@100.80.36.111
```

Puis dans le navigateur : <https://localhost:9443>



7. Accès aux services applicatifs

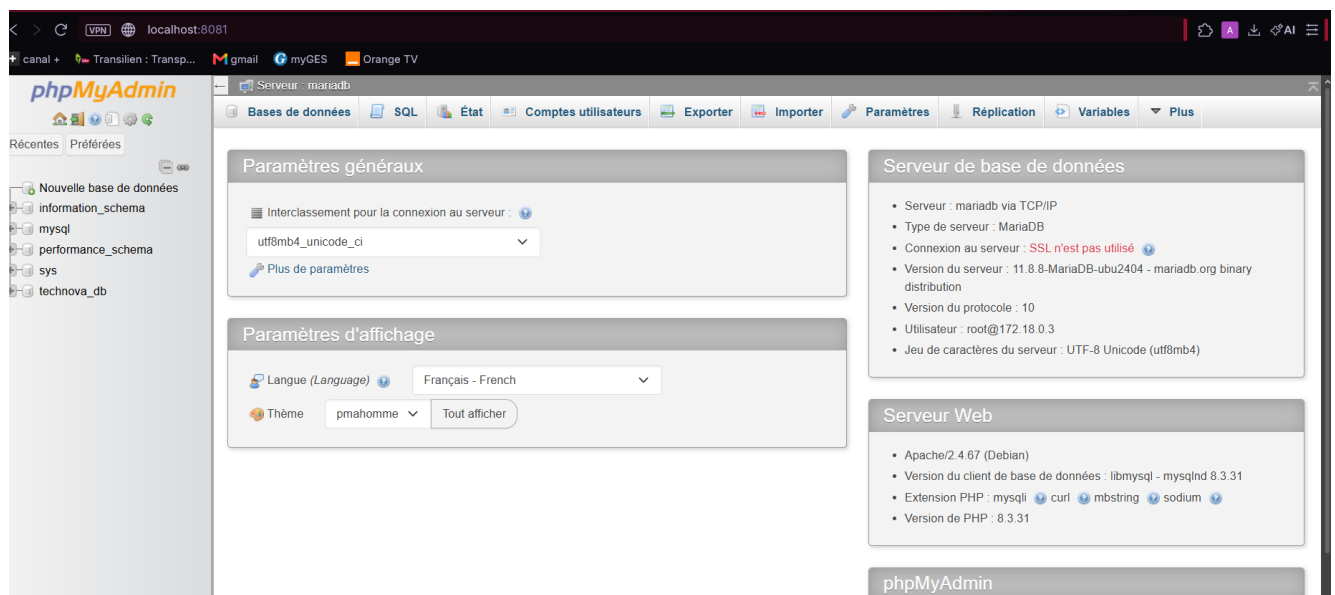
Les services web (nginx, phpMyAdmin, Portainer) ne sont jamais exposés directement. L'accès depuis le poste admin passe par un tunnel SSH multi-ports :

```
ssh -p 2222 \
  -L 8080:192.168.20.30:8080 \
  -L 8081:192.168.20.30:8081 \
  -L 9443:192.168.20.30:9443 \
  technova@100.80.36.111
```

URLs accessibles depuis le navigateur du poste admin :

- <http://localhost:8080> — Page nginx TechNova
- <http://localhost:8081> — phpMyAdmin (login root / TechNova2026!)

- <https://localhost:9443> — Portainer CE



8. Justification des choix techniques

8.1 Docker Engine vs Podman

Docker Engine officiel a été choisi pour :

- Maturité (12 ans d'existence)
- Compatibilité totale avec docker-compose.yml standard
- Large adoption entreprise / écosystème CI/CD
- Documentation exhaustive

8.2 Docker Compose vs docker run

Docker Compose est utilisé pour tous les déploiements applicatifs :

- Infrastructure as Code (YAML versionnable dans Git)
- Documentation vivante des services
- Gestion native des dépendances (depends_on)
- Redéploiement reproductible en une commande
- Standard industrie DevOps

8.3 nginx Alpine + MariaDB + phpMyAdmin

nginx Alpine :

Image légère (~7 Mo), surface d'attaque minimale

MariaDB 11 :

Fork open-source de MySQL, activement maintenu, licence GPLv2

phpMyAdmin :

Interface d'administration standard, communauté active

9. Sécurité

- Aucun port Docker exposé sur Internet (accès Bastion uniquement)
- Compte technovaadmin distinct de root et de technova
- Groupe docker limité aux comptes admins
- Secrets externalisés dans .env (chmod 600, .gitignore)
- Réseau Docker isolé (webapp_technova bridge)
- MariaDB non exposée à l'hôte (pas de port mapping)